

Sprawozdanie z laboratorium: Układy równań

Vladyslav Humeniuk

1 Wstęp

Celem niniejszego ćwiczenia była empiryczna weryfikacja teoretycznej złożoności obliczeniowej algorytmu rozwiązywania układów równań liniowych postaci $Ax = b$ z wykorzystaniem dekompozycji LU. Literatura przedmiotu wskazuje, że rozkład nieosobliwej macierzy A na macierz dolnotrójkątną (L) i górną (U) wymaga wykonania około $\frac{2}{3}N^3$ operacji zmiennoprzecinkowych, co przekłada się na złożoność czasową rzędu $O(N^3)$ [4]. Natomiast drugi etap procesu, czyli rozwiązanie układu metodą podstawiania (w przód i wstecz), charakteryzuje się znacznie niższą złożonością obliczeniową.

W sprawozdaniu przedstawiono analizę wydajności obu etapów algorytmu przy użyciu biblioteki *GNU Scientific Library* (GSL)[1]. Pomiary czasu wykonano dla macierzy o wymiarach z przedziału od $N = 10$ do $N = 1000$.

2 Metodyka

Program testowy zaimplementowano w języku C, wykorzystując funkcje biblioteczne GSL: `gsl_linalg_LU_decomp` oraz `gsl_linalg_LU_solve`, zgodnie z dokumentacją[1]. Procedura badawcza obejmowała następujące kroki:

1. **Generacja danych:** Dla każdego badanego wymiaru $N \in [10, 1000]$, macierz A oraz wektor b wypełniono losowymi wartościami z wykorzystaniem struktur wektorów i macierzy GSL[3].
2. **Dekompozycja:** Zmierzono czas operacji faktoryzacji LU macierzy A .
3. **Rozwiązywanie:** Zmierzono czas wyznaczenia wektora wynikowego x na podstawie wcześniejszego rozkładu.
4. **Weryfikacja:** Poprawność wyniku sprawdzono poprzez obliczenie normy euklidesowej wektora residuum $r = \|Ax - b\|$, wykorzystując operacje BLAS poziomu 2 dostępne w GSL[2, (author?) [5]].

Do pomiaru czasu wykorzystano funkcję systemową `getrusage()`, sumującą czas użytkownika i systemowy, co pozwoliło na uzyskanie precyzji mikrosekundowej.

3 Wyniki

Zestawienie uzyskanych czasów wykonania oraz błędów numerycznych zaprezentowano w Tabeli 1. Wartości błędu residuum dla wszystkich prób oscylowały w granicach precyzji maszynowej (10^{-9} do 10^{-14}), co potwierdza poprawność i stabilność numeryczną zastosowanych metod[1].

Table 1: Czasy wykonania i błędy rezydualne dla wybranych rozmiarów macierzy.

N	Decomposition Time [s]	Solving Time [s]	Error $\ Ax - b\ $
10	0.000024	0.000009	3.2171×10^{-14}
100	0.000875	0.000049	4.7298×10^{-12}
500	0.031188	0.000335	5.7555×10^{-10}
600	0.057770	0.000595	8.4344×10^{-11}
800	0.117500	0.000861	3.4065×10^{-10}
1000	0.234349	0.001321	1.0893×10^{-09}

3.1 Analiza złożoności obliczeniowej

Otrzymane wyniki jednoznacznie ukazują dysproporcję w tempie wzrostu kosztu obliczeniowego obu operacji. Na Rysunku 1 przedstawiono czas dekompozycji, który wykazuje charakterystyczny dla złożoności $O(N^3)$ trend sześcienny, zgodny z teorią numeryczną[4].

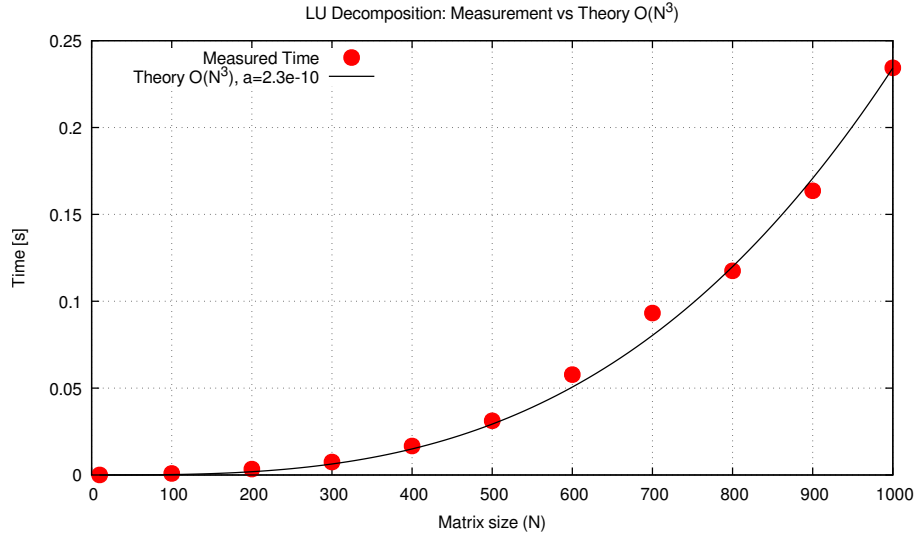


Figure 1: Zależność czasu dekompozycji LU od rozmiaru macierzy N . Widoczny trend wzrostu rzędu $O(N^3)$.

Z kolei czas rozwiązywania układu (Rysunek 2) rośnie w tempie kwadratowym. Nawet dla dużych wartości N , operacja ta pozostaje rzędu wielkości szybsza niż sama faktoryzacja.

Aby precyzyjnie ocenić skalowalność algorytmu, porównano wzrost czasu wykonania przy dwukrotnym zwiększeniu rozmiaru problemu (z $N = 500$ do $N = 1000$):

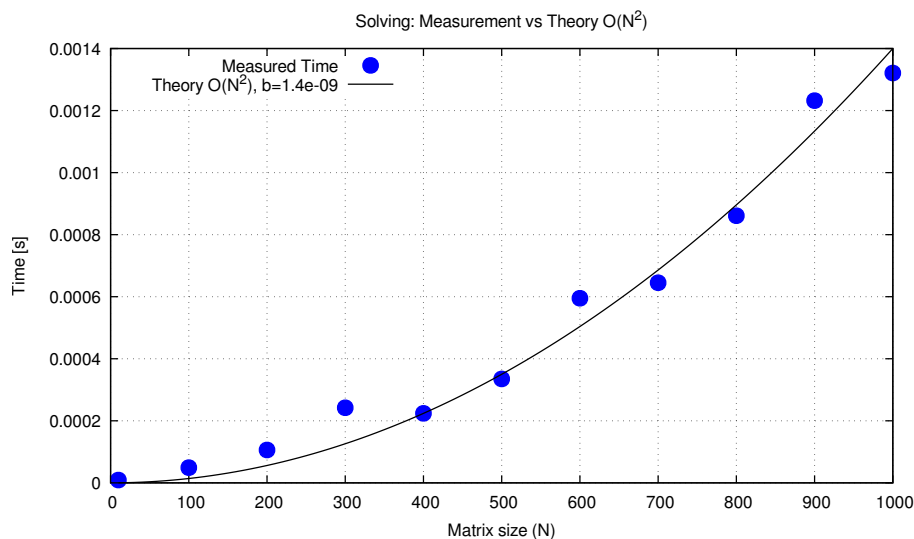


Figure 2: Zależność czasu rozwiązywania układu od rozmiaru macierzy N . Trend zgodny ze złożonością $O(N^2)$.

1. Etap dekompozycji:

$$\frac{T_{decomp}(1000)}{T_{decomp}(500)} = \frac{0.234349}{0.031188} \approx 7.51$$

Teoretycznie, dla złożoności sześcienniej oczekiwany współczynnik wzrostu wynosi $2^3 = 8$. Uzyskany wynik (ok. 7.51) jest bardzo bliski wartości teoretycznej, co potwierdza sześcienną naturę algorytmu. Niewielkie odchylenie wynika prawdopodobnie z narzutów systemowych i zarządzania pamięcią, które mają relatywnie większy wpływ przy mniejszych wymiarach macierzy.

2. Etap rozwiązywania:

$$\frac{T_{solve}(1000)}{T_{solve}(500)} = \frac{0.001321}{0.000335} \approx 3.94$$

Wynik ten jest niemal identyczny z wartością teoretyczną dla złożoności kwadratowej ($2^2 = 4$).

Rysunek 3 w skali logarytmicznej obrazuje powiększającą się różnicę między nakładem obliczeniowym obu faz procesu.

4 Wnioski

Przeprowadzone eksperymenty w pełni potwierdzają teoretyczne założenia dotyczące złożoności algorytmów algebry liniowej omówione w materiałach

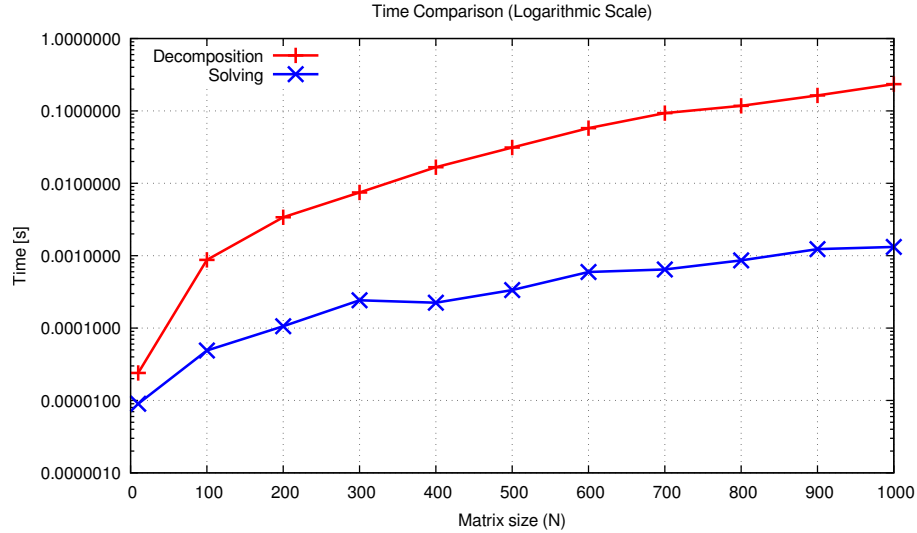


Figure 3: Porównanie czasów dekompozycji i rozwiązywania w skali logarytmicznej. Różne nachylenia krzywych odpowiadają różnym klasom złożoności.

dydaktycznych[4]. Koszt dekompozycji LU ($O(N^3)$) dominuje nad kosztem rozwiązania układu ($O(N^2)$), co staje się szczególnie widoczne wraz ze wzrostem wymiaru zadania.

Dla $N = 1000$, proces dekompozycji trwał około 177 razy dłużej niż samo wyznaczenie wyniku. Uzasadnia to powszechnie stosowaną strategię numeryczną: w przypadku konieczności rozwiązania wielu układów z tą samą macierzą główną A , ale różnymi wektorami wyrazów wolnych b , najbardziej efektywnym podejściem jest jednokrotne wykonanie dekompozycji, a następnie wielokrotne wykorzystanie uzyskanej faktoryzacji do szybkiego ($O(N^2)$) wyznaczania rozwiązań.

Bibliography

- [1] GNU Project: Algebra liniowa w gsl. <https://www.gnu.org/software/gsl/doc/html/linalg.html> (2026), dostęp: 2026-02-17
- [2] GNU Project: Biblioteka gsl cblas. <https://www.gnu.org/software/gsl/doc/html/blas.html> (2026), dostęp: 2026-02-17
- [3] GNU Project: Wektory i macierze w gsl. <https://www.gnu.org/software/gsl/doc/html/vectors.html> (2026), dostęp: 2026-02-17
- [4] Guidry, M.: Dekompozycja lu. http://csep10.phys.utk.edu/guidry/phys594/lectures/linear_algebra/lanotes/node3.html (2026), materiały dydaktyczne

- [5] Guidry, M.: Operacje na wektorach i macierzach - blas. http://csep10.phys.utk.edu/guidry/phys594/lectures/linear_algebra/blas.html (2026), materiały dydaktyczne